

# Sentinelleai — Audit Report

IMagnetaOFTStandardFactory.sol · 29db593d-900f-423d-9e07-d64fc2a3dde7 ·  
2026-05-22 11 h 20 min 13 s

## **FAIL** 38 / 100 5 consolidated findings

The CreateTokenDispatcher contract presents a **CRITICAL** vulnerability (SC05 — Insecure Cross-Chain Messaging) in its `_IzReceive` implementation, which lacks explicit runtime validation of `_origin.sender` against the registered peer for `_origin.srcEid`. Despite the developer comment claiming peer wiring provides protection, no on-chain enforcement exists inside the override, meaning any contract on any chain can forge `CREATE_TOKEN` messages and spawn tokens on behalf of arbitrary creators — directly mirroring the Kelp DAO DVN-spoofing incident (\$292M, 2026-04-19). A **HIGH**-severity replay concern (SC05) is partially mitigated by LayerZero V2's guid deduplication at the endpoint level, but the contract provides no additional on-chain nonce or processed-message map, leaving it exposed to the 2026 replay-semantic divergence attack class. A **MEDIUM** finding (SC01 — Improper Input Validation) exists for missing zero-address checks on the creator parameter passed to factory calls. Two **LOW** findings (SC02 — Denial of Service / Resource Exhaustion) relate to permissionless factory interfaces lacking rate-limiting or registry guards. The **CRITICAL** trusted-sender gap alone warrants a **FAIL** verdict; the contract must not be deployed to production without adding the peer-sender assertion in `_IzReceive`.

## Severity breakdown

1 **CRITICAL**   1 **HIGH**   1 **MEDIUM**   1 **LOW**   1 **INFO**

## Static sentinel pre-scan

Static sentinels matched 5 pattern(s) — 1 CRITICAL, 2 MEDIUM, 2 LOW.

**CRITICAL** SC05 · LZ-DVN-1 · line 201

### IzReceive without trusted-sender check

IzReceive / \_IzReceive must validate \_origin.sender against peers[\_origin.srcEid]; a missing check lets any chain forge messages.

```
function _IzReceive( Origin calldata _origin, bytes32 /*_guid*/, bytes calldata
_payload, address /*_executor*/, bytes calldata /*_extraData*/ ) internal override
whenNotPaused { if (_payl
```

**MEDIUM** SC01 · FACT-2 · line 11

### Factory accepts address parameters without zero-address check

create\*/deploy\* functions that store `address` parameters (token, recipient, manager) without `require(\_x != address(0))` brick the resulting pool — token transfers revert, fees go to the zero address, etc.

```
function createForCreator – unchecked: creator
```

**LOW** SC02 · FACT-5 · line 11

### Permissionless create\* function with no rate-limit or registry guard

Open `createPool/createPair/deploy\*` is intentional for some launchpad designs, but it should at minimum emit a Creator-indexed event AND track the caller — otherwise attackers can grief gas/storage by spamming pools. Flag for review.

```
function createForCreator( address creator, string memory name, string memory symbol,
string memory tokenURI, uint256 totalSupply, bool revokeUpdate, bool revokeFreeze,
bool revokeMint ) external retu
```

**LOW** SC02 · FACT-5 · line 24

### Permissionless create\* function with no rate-limit or registry guard

Open `createPool/createPair/deploy\*` is intentional for some launchpad designs, but it should at minimum emit a Creator-indexed event AND track the caller — otherwise attackers can grief gas/storage by spamming pools. Flag for review.

```
function createForCreator( address creator, string memory name, string memory symbol,
string memory tokenURI, uint256 totalSupply, uint256 liquidityToBurn ) external
returns (address); } /// @title Cr
```

**MEDIUM** SC05 · RELAY-1 · line 201

### Cross-chain message handler without nonce / replay tracking

Functions that consume a cross-chain payload (handle/receiveMessage/IzReceive) must dedupe with a nonce or processed-message map. The new 2026 attack class is replay-semantic divergence between chains.

```
function _IzReceive( Origin calldata _origin, bytes32 /*_guid*/, bytes calldata
_payload, address /*_executor*/, bytes calldata /*_extraData*/ ) internal override
whenNotPaused { if (_payl
```

## Consolidated findings

**CRITICAL** CVSS 9.8 SC05: Insecure Cross-Chain Messaging

### **lzReceive lacks trusted-sender / peer validation — forged cross-chain token creation**

\_lzReceive decodes and executes the payload without verifying that \_origin.sender equals peers[\_origin.srcEid]. The developer comment states 'peer wiring ensures only sibling CreateTokenDispatchers can deliver here', but this is not enforced at runtime inside the override. Any contract on any supported chain can craft a payload, set an arbitrary creator address, and cause the dispatcher to call createForCreator on the local factory, minting tokens on behalf of any EOA. This matches the Kelp DAO DVN-spoofing pattern (\$292M, 2026-04-19) and the static sentinel LZ-DVN-1 finding.

CreateTokenDispatcher.\_lzReceive (line 201)

→ Add an explicit check at the top of \_lzReceive: require(peers[\_origin.srcEid] != bytes32(0) && peers[\_origin.srcEid] == \_origin.sender, 'CTD: untrusted sender'). Verify whether the OApp base class already enforces this in its internal routing; if so, document the inherited protection explicitly.

Confirmed by: static-sentinel, developer, historian, formalist

**HIGH** CVSS 7.5 SC05: Insecure Cross-Chain Messaging

### **Cross-chain create handler relies solely on LZ guid for replay protection — no on-chain nonce map**

The contract processes incoming payloads based entirely on LayerZero V2's endpoint-level guid deduplication. No on-chain processed-guid or per-origin nonce map is maintained. The 2026 replay-semantic divergence attack class (CrossCurve, \$3M, 2026-02) demonstrates that endpoint-level deduplication alone is insufficient when chains can diverge in state or when endpoint upgrades alter guid semantics. A compromised or upgraded endpoint could replay messages.

CreateTokenDispatcher.\_lzReceive / \_executeCreate (line 201+)

→ Maintain a mapping(bytes32 ⇒ bool) processedGuids and set processedGuids[\_guid] = true before executing \_executeCreate, reverting if already set. This adds defense-in-depth beyond the LZ endpoint guarantee.

Confirmed by: static-sentinel, historian, formalist

**MEDIUM** CVSS 5.3 SC01: Improper Input Validation / Access Control

### Factory createForCreator accepts zero-address creator without validation

The createForCreator functions in both IMagnetaOFTStandardFactory and IMagnetaOFTAutoLiquidityFactory interfaces accept a creator address parameter with no zero-address guard. If a malformed or adversarial payload decodes to creator == address(0), the factory will mint tokens owned by the zero address, permanently locking ownership and any associated fees or governance rights. The static sentinel FACT-2 and multiple agents confirm this pattern.

```
IMagnetaOFTStandardFactory.createForCreator (line 11);  
IMagnetaOFTAutoLiquidityFactory.createForCreator (line 24); _createStandard  
/ _createAutoLiquidity
```

→ Add require(creator != address(0), 'CTD: zero creator') in \_createStandard and \_createAutoLiquidity before calling the factory, and ensure factory implementations also validate this parameter.

Confirmed by: static-sentinel, developer, historian, formalist

**LOW** CVSS 3.1 SC02: Denial of Service / Resource Exhaustion

### Permissionless cross-chain token creation with no rate-limit or spam guard

fanOutCreate is callable by any address with sufficient ETH for LZ fees. There is no per-caller rate limit, cooldown, or registry guard. An attacker can spam token creation across all 19 chains, bloating factory storage and event logs, and potentially exhausting gas budgets on destination chains. The static sentinel FACT-5 flags both factory interfaces for this pattern.

```
CreateTokenDispatcher.fanOutCreate (line 153); IMagnetaOFTStandardFactory  
(line 11); IMagnetaOFTAutoLiquidityFactory (line 24)
```

→ Consider adding a per-caller cooldown (e.g., mapping(address ⇒ uint256) lastCreateTime with a minimum interval), a whitelist, or a creation fee burned to a treasury to economically deter spam.

Confirmed by: static-sentinel, developer, formalist

**INFO** CVSS 1.5 SC05: Insecure Cross-Chain Messaging

### Replay protection security assumption not documented — reliance on LZ V2 endpoint

The contract explicitly delegates replay protection to LayerZero V2's guid mechanism (comment at line 200). This is a valid design choice for standard OApp contracts, but the assumption is not formally documented in NatSpec or a security model document, making it easy for future maintainers to miss the dependency.

CreateTokenDispatcher.\_lzReceive comment (line 200)

→ Add a NatSpec @dev note explicitly stating the replay-protection invariant and the LayerZero endpoint version dependency. Consider adding an integration test that verifies guid-based deduplication at the endpoint level.

Confirmed by: developer

## Historical precedents

Retrieved from the local bug-bounty corpus (~20 700 findings) by vulnerability-class match.

**CRITICAL** bridge ethereum 2022-03-29 \$624 000 000

Ronin Network — Bridge (2022-03-29)

<https://rekt.news/ronin-rekt/>

**CRITICAL** bridge ethereum 2021-08-11 \$611 000 000

Poly Network — Bridge, getting around modifier through cross-chain message (2021-08-11)

<https://rekt.news/polynetwork-rekt/>

**CRITICAL** bridge bsc 2022-01-28 \$80 000 000

Qubit Finance — Bridge address(0).safeTransferFrom() does not revert (2022-01-28)

<https://rekt.news/qubit-rekt/>

**MEDIUM** bm25 2023-04-01

rubicon — Anyone can create buddy for himself without calling

`BathHouseV2.createBathToken` function

<https://github.com/code-423n4/2023-04-rubicon-findings/issues/1172>

**MEDIUM** bm25 2023-05-01

maia — RestakeToken function is not permissionless

<https://github.com/code-423n4/2023-05-maia-findings/issues/534>

**MEDIUM** bm25 2024-04-01

noya — Liquidity position is removed from registry without checking that full liquidity has been removed from it.

<https://github.com/code-423n4/2024-04-noya-findings/issues/856>

**MEDIUM** bm25 2024-08-01

phi — `RewardControl.withdrawFor` function can be called without `from` address' authorization, which can frontrun and DOS `from` address' `RewardControl.withdraw` function call and trigger `from` address' `receive` function's logics without its approval

<https://github.com/code-423n4/2024-08-phi-findings/issues/35>

**MEDIUM** bm25 2022-09-01

nouns-builder — Anyone can create a proposal without any voting power

<https://github.com/code-423n4/2022-09-nouns-builder-findings/issues/4>

**HIGH** bm25 2023-07-01

tapioca — `\_claimRewards` function lack a proper reentrancy guard, without properly handling the execution flow, it becomes susceptible to attacks.

<https://github.com/code-423n4/2023-07-tapioca-findings/issues/948>

**HIGH** bm25 2023-07-01

tapioca — user can withdraw collateral without check making complete without complete repayment

<https://github.com/code-423n4/2023-07-tapioca-findings/issues/892>

**MEDIUM** bm25 2024-01-01

curves — In the `FeeSplitter.sol#onBalanceChange` function, `token` is `push`ed to `userTokens` without performing a duplicate check.

<https://github.com/code-423n4/2024-01-curves-findings/issues/1201>

**HIGH** bm25 2024-01-01

curves — FeeSplitter.setCurves is permissionless

<https://github.com/code-423n4/2024-01-curves-findings/issues/508>

---

Generated by Sentinelleai on 2026-05-22T15:38:56.163Z. Audit ID: 29db593d-900f-423d-9e07-d64fc2a3dde7.

This report combines deterministic regex+AST sentinels, optional Slither/Mythril static analysis, and a 6-model adversarial LLM panel consolidated by a Judge agent.